Delphi Project Handoff Engineering Guide

Build maintain and validate the learning application

Experimental Windows desktop software

Last changed 13 October 2026

Read in order for your first session. Later, use the contents to find a particular procedure.

Contents

Purpose and architecture....3
Source and storage boundaries....4
Native interface and lesson generation....5
Build and run DPH....6
Validate a change....7
Package and deploy....8
Publication and privacy....9
Maintain the learning product....10

Purpose and architecture

The learning contract

Delphi Project Handoff (DPH) is an independent FireMonkey desktop application for learning one selected Delphi application or project group. Its primary interface presents purpose, screens, actions, guided tasks and attributed answers. Source analysis supports that learning route. Preparation detail should answer a question or help produce reliable lessons.

Main responsibilities

DPH.UI.Main.pas and its .fmx resource own navigation and native editors. DPH.Workspace.Worker.pas performs background capture, opening, saving and publication. DPH.Project.Analysis.pas orchestrates bounded read-only inventory and parsing. DPH.Analysis.Symbols.pas resolves supported relationships. DPH.Storage.Database.pas persists coherent source generations and immutable authored revisions.

DPH.Workspace.Model.pas provides the captured view and typed task contract. DPH.Analysis.Explanations.pas generates beginner topics and object explanations from selected facts. DPH.Publish.Model.pas builds scoped, redacted publication data. DPH.Publish.Export.pas renders learning pages, technical indexes and editable guides. DPH.Publish.BrowserPolicy.pas restricts local navigation.

Maintain the supplied practice lessons

samples\OrderPractice contains a deliberately small FMX application and its prepared lessons. Try practice example reads this source and imports the packaged nineteen attributed records into its own database. It never runs the example. The first import checks every declared source and picture hash and resolves each requested source link to exactly one captured object. Reopening uses the saved database and preserves edited lessons.

DPH.Tests.OrderSample validates the example calculation and captures the actual native form. DPH.Tests.Practice exercises the beginner route through DPH, the four ordered tasks, fourteen explanations, two images, source links, progress and shared edition. Only the designated sample test runs sample code; passive discovery remains read-only.

Data route

Selection leads to bounded input acquisition, syntax and relationship analysis, then coherent database publication. Captured facts and authored knowledge feed explanations. Native readers and exported pages use that material. Human instructions survive source reassessment.

Source and storage boundaries

Read without executing the selected application

Discovery follows the selected project and permitted dependencies. It reads source and form resources without compiling the target, loading its component classes, running scripts or contacting live services. Missing input and unsupported syntax produce diagnostics. Supported source relationships remain distinguishable from uncertain or inferred behavior.

Persist reliable lessons

Each selected project has a .dphdb SQLite store accessed through FireDAC, with numbered migrations under migrations. The workspace catalog remembers locations and preferences separately. Managed .dphdb.views material travels beside its database. Generation publication is atomic; failure or interruption preserves the last usable generation.

SaveKnowledge uses expected revision values to prevent stale editors overwriting newer content. Edits and reviews create immutable history. learning_lesson records have a stable identity, six named fields and ordered action/expected_result steps. Storage validates their canonical body, contributor, scope and identity. Legacy free text retains its original instructions.

A lesson may include a learning object with role, path and integer order. The native reader filters by role/path and sorts the prepared sequence. Personal checked-step progress lives in the separate workspace catalog, keyed by database, learner, lesson identity and exact authored revision. A new instruction revision cannot inherit completion.

learning_card records contain a stable card identity, a teaching category and seven named fields. A canonical body agrees with those fields. Optional picture metadata identifies a previously imported image and a point normalized from 0 to 10000 on each axis. Save rejects nonexistent project pictures and out-of-bounds points. Native and HTML overlays preserve the original PNG bytes.

Retain truthful explanations

Hashes and source ranges identify evidence. They do not establish a successful run or a correct business rule. Rendering a lesson or hiding a page must not clear partial coverage. Preserve pending contributions and Unknown fields until someone supplies or verifies the missing knowledge.

Native interface and lesson generation

Keep forms editable

Open DPH.UI.Main.fmx and dedicated dialog forms in RAD Studio. Change layout, captions, margins, fonts and control properties in the Object Inspector. Ordinary forms must use serialized controls. HTML is a generated view hosted by serialized browser controls, with native text where supported.

Separate learning and preparation

LessonPageCombo contains learner destinations. SetupTab holds preparation, with source settings collapsed in AnalysisSettingsExpander. WorkspacePageChange builds optional reports on demand. Opening lessons should not build every report for hidden pages.

TaskTab presents saved instructions one step at a time. TaskEditorTab holds fields and the ordered draft. Keep step accepts a step edit; Save task persists the whole record through the worker and returns to the reader. Protect drafts during opening and saving. A failed save must retain entered instructions.

CardLibraryTab reads and searches teaching explanations; CardEditorTab edits their seven instructional fields. Save explanation persists a complete record through the same worker. ScreenshotMarkers draws numbered callouts using the serialized marker style. TaskContextExpander keeps prerequisites available while leaving room for the current action and result in a compact window.

Generate from selected facts

LessonHTML supplies twelve topics. ObjectLessonHTML introduces the selected object, linked human purpose, actual choices and bounded next connections. The source walkthrough remains optional. The native text fallback uses the same lesson. Unlinked global notes must not masquerade as the purpose of every object.

Build and run DPH

Development requirements

Use Windows, RAD Studio 37.0 with Delphi and FireMonkey, FireDAC SQLite support and Microsoft WebView2 Runtime for embedded pages. Build scripts initialize the compiler environment through rsvars.bat. Debug and Win32 is the validated desktop configuration; other builds need their own validation.

Build from source

1. Open an elevated Windows command prompt so RAD Studio can read its environment settings.

2. Change to the DPH source root containing DelphiProjectHandoff.dproj and tools.

3. Run tools\build-wp0.cmd. It builds under bin\Win32\Debug and runs the serialized FMX resource and binding baseline.

4. Require a zero exit code and PASS WP0-TEST. Investigate the first error if compilation fails.

5. Start bin\Win32\Debug\DelphiProjectHandoff.exe from the source tree. Keep WebView2Loader.dll beside it; its migration search locates the supplied migrations folder.

Use the IDE

Open DelphiProjectHandoff.dproj, select Debug and Win32 and inspect the main form before building. If a running DPH instance locks the output, check for unsaved edits, close that instance and rebuild. Do not close unrelated applications to solve a build failure.

Validate a change

Run meaningful checks

Native regressions instantiate serialized forms and use real WebView2 rendering. Persistence checks use disposable databases. Test fixtures are copies or designated samples; selected application sources must remain unchanged. Automated checks do not constitute independent beginner acceptance.

1. Run tools\build-wp0.cmd for the application and form baseline.

2. Run tools\build-learning-refocus.cmd. Its optional argument is a prepared reference .dphdb; omit it for the small-fixture checks.

3. Run tools\build-guided-tasks.cmd followed by a new disposable database path outside the source fixture folder. It checks ordering, saving, reopening and task export.

4. Run tools\build-wp6.cmd for publication, redaction, navigation and integrity. Run tools\build-phase1.cmd when changing the project profile.

5. Read final PASS lines and process exit codes. Resolve failures before committing. Add the appropriate analyzer or storage suite when those contracts change.

Check the learning route

Open a representative application. Read its purpose, follow a screen event, inspect a decision and read a prepared task. Save and reopen an answer. Export and follow the same object from learning.html. Check maximized and restored windows for readable labels, scrolling and expected results.

Contain disposable output

Keep validation output under bin in clearly designated folders. Retain failure evidence until diagnosed, then remove obsolete generated workspaces. Preserve one reusable reference cache where useful. Never delete source fixtures, owner databases or authored material while cleaning tests.

Package and deploy

Create the desktop folder

The package needs DelphiProjectHandoff.exe, WebView2Loader.dll, migrations, help, current guides, Databases instructions and samples\OrderPractice with its lesson pack and pictures. DPH.ValidateHandoff.exe is an included technical validation tool. Microsoft WebView2 Runtime is a system prerequisite separate from its loader DLL.

1. Build and validate the application and publication validator.

2. Run tools\package-dph.ps1 through PowerShell with ExecutionPolicy Bypass. It creates a new versioned staging folder and refuses an existing one.

3. Verify the checksum report and extract the ZIP into a different local folder.

4. Start the extracted executable and confirm Getting started, native controls and migration discovery. Use only a disposable database for this deployment check.

5. Check the package contains current guides and help, with no owner databases, private documents, development history or personal path references.

Give recipients plain directions

Tell recipients to extract the package, open its folder and double-click DelphiProjectHandoff.exe. The User Guide describes deployed files and visible controls. Compiler configuration belongs in this Engineering Guide. Describe source locations with relative project paths.

Publication and privacy

Publish learning first

learning.html combines twelve topics, supplied tasks, answers, images and business terms. Hash-named object pages begin with beginner explanations and retain optional source detail. index.html and JSONL files support technical inspection. Native Preview lessons opens learning.html.

Render from a scoped, redacted TDPHDocumentModel. Filter excluded files and unavailable links before rendering, and encode contributed prose as text. Screenshots remain contributor-supplied material. Source navigation and generated agent instructions require explicit options.

Validate an edition

Run DPH.ValidateHandoff.exe with the edition folder and a separately retained trusted input fingerprint. The default rejects stale or partial material. --allow-stale permits historical integrity inspection without changing that status. The checksum index detects modifications; it does not authenticate an unknown author.

Create PDF companions

tools\export-pdf.ps1 accepts -BundleDirectory for an edition. It uses LibreOffice, populates changeable contents fields and refreshes checksums. tools\build-guides.py rebuilds DPH's manuals and matching HTML help. Inspect every rendered page after a layout-sensitive edit.

Maintain the learning product

Plan a focused change

Begin with a learning problem: an unclear task, an unexplained control or an unanswered question. Locate the native form and responsible model. Reuse save and publication contracts rather than creating another administrative system. Keep the learner route concise and supporting detail optional.

Protect the data

Take the required verified backup before approved changes and synchronize the configured Git repositories after validated commits. Development backup locations belong to the environment, not the User Guide. For storage or migration changes, verify old databases open and authored revisions survive.

Investigate failures

For preparation failure, inspect the first diagnostic, missing files, scope and intended defines. For a save conflict, reopen the latest revision and reconcile the contribution. For a browser failure, check WebView2 Runtime and loader availability and use native text. For broken sharing links, regenerate from the scoped model and validate the edition.

Maintain the manuals

Update instructions from serialized controls, source and observed behavior. Use real screenshots with clear example captions. Remove obsolete names and personal paths. Refresh the contents and PDF, inspect every page and package only current guides. Keep requirement audits and test history separate from end-user instruction.